

Assignment 1: Review of Related Work and Search Engine Design

1. Introduction

Modern information retrieval (IR) systems must accommodate diverse user intents, ranging from exact known-item searches to exploratory semantic queries. Users search for artworks via precise identifiers (e.g., *The Weeping Woman*) or through conceptual descriptions (e.g., *a gloomy landscape*). Traditional keyword systems fail the latter due to vocabulary mismatch. This project engineers a hybrid retrieval architecture, integrating sparse probabilistic matching with dense vector embeddings to support both lexical precision and semantic recall while maintaining low latency.

2. Literature Review

2.1 Probabilistic Keyword Retrieval and Term Weighting

Standard term-frequency approaches often incorrectly rank long documents higher by chance. Robertson et al. (1994) addressed this with the BM25 retrieval function, incorporating non-linear term frequency saturation and document length normalization. While BM25 is highly efficient for exact matching, it fundamentally fails to capture semantic meaning.

2.2 Extending BM25 to Structured Fields

BM25 was designed for unstructured text. Computing separate scores for structured fields (titles, artists) and combining them linearly breaks its non-linear saturation properties. Robertson, Zaragoza, and Taylor (2004) proposed BM25F, which linearly combines term frequencies across fields *before* applying the saturation function, weighting terms by field importance. Despite this, BM25F remains limited by exact keyword matching.

2.3 Deep Semantic Understanding and Dense Retrieval

Devlin et al. (2019) introduced BERT to capture deep, bidirectional word context, solving the vocabulary mismatch problem. However, real-time cross-attention is computationally unfeasible for search. Karpukhin et al. (2020) resolved this with Dense Passage Retrieval (DPR), utilizing a dual-encoder to independently embed documents and queries into a shared low-dimensional space. Retrieval is then executed via mathematical vector similarity, bridging the semantic gap efficiently.

2.4 Literature Review Conclusion

The literature reveals a clear dichotomy: sparse models (BM25F) excel at efficient, exact keyword matching but fail on conceptual queries due to vocabulary mismatch. Conversely,

dense models (BERT/DPR) bridge this semantic gap but incur high computational costs. To resolve this, the proposed Hybrid Architecture fuses BM25F precision with DPR-style semantic embeddings, maximizing the strengths of both paradigms while neutralizing their individual limitations.

3. Task & Dataset

The system interfaces with a randomly sampled corpus of 2,000 digitized art gallery records. This scale justifies the use of exhaustive, exact retrieval algorithms without the accuracy trade-offs of approximate search structures. Documents contain *Title*, *Artist*, and *Description* metadata.

The system is designed to navigate two primary Information Retrieval (IR) scenarios:

- **Known-Item Retrieval:** A navigational task where the system must identify a specific, unique entity based on precise identifiers (e.g., searching for The Weeping Woman by Pablo Picasso). The primary performance target is a Reciprocal Rank of 1.0, indicating the target document is retrieved at the first position.
- **Ad-Hoc Semantic Retrieval:** An informational task focused on thematic and conceptual discovery (e.g., a gloomy landscape with swirling clouds). This task requires the engine to bridge the vocabulary mismatch - matching query concepts to document descriptions even when no exact keyword overlap exists.

4. General Architecture of the Search Engine

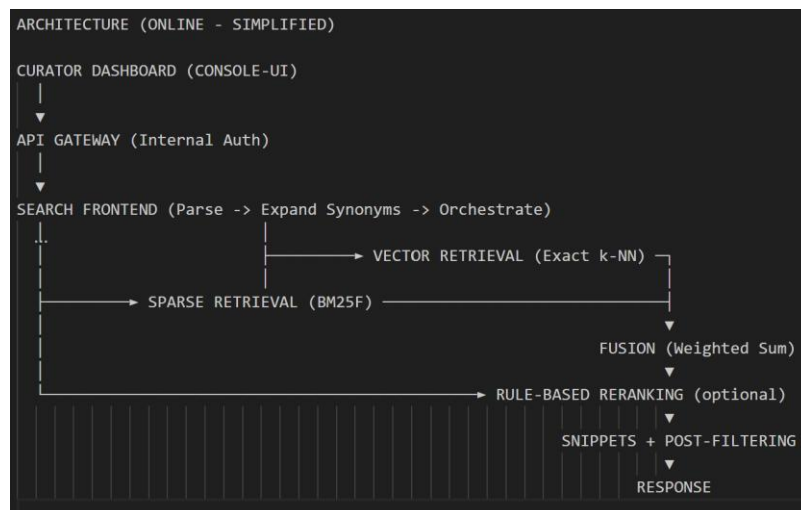


Figure 1. diagrams / architecture_overview.png

4.1 Key Architecture Decisions

- **Hybrid Retrieval:** A dual-path strategy is employed: Sparse (BM25F for exact title/artist matching) and Dense (BERT embeddings for semantic matching).

- **Simplified Stack:** The system runs as a local Python service on a single node, utilizing exact k -NN.
- **Transparent Ranking:** The system uses Rule-Based Boosting (e.g., Title Match = 1.5x multiplier).

4.2 Layers and Responsibilities

The system is a local desktop application with no web UI, API, or network dependencies.

- **Client:** A local GUI/CLI accepting direct user input and displaying results.
- **Search Core:** Orchestrates query parsing, pipeline execution, and result fusion.
- **Retrieval Services:** Houses the Sparse module (BM25F inverted index) and the Vector module (Exact k -NN over embeddings).

An optional post-fusion reranking stage finalizes the ranked list with contextual snippets.

4.3 Ingestion and Document Processing

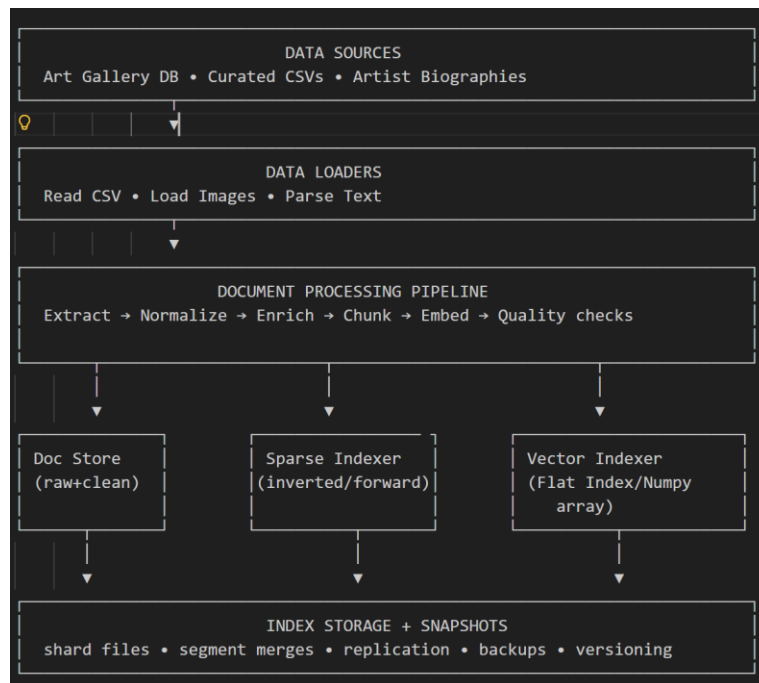


Figure 2. diagrams / ingestion/indexing.png

4.3.1 Data Sources

- **Primary Sources (Internal):** Art Collection Database (Metadata); Curatorial Descriptions (Text); Digital Images (Visuals).
- **Secondary Sources (External Enrichment):** Gallery Websites (Web Scraping) targeting external museum pages that contain supplementary information relevant to the internal collection.

4.3.2 Processing Workflow

A linear Batch Processing Script is used:

- **Load:** Read the CSV catalogue and load associated text descriptions.
- **Pre-process:** Apply text normalisation and tokenization.
- **Enrich:** Pass descriptions through the BERT model to generate dense vector embeddings.
- **Index:** Build the Sparse Index (Inverted Index dictionary) and Vector Index (NumPy array).
- **Save:** Serialize the indices to disk.

4.3.3 Extraction and Normalisation

Simple text extraction logic is used; structured data is read from CSV, while unstructured text is extracted from files and web content (BeautifulSoup). Standard pipeline includes lowercasing, accent removal, tokenization, stop-word removal, and stemming (Porter Stemmer).

4.3.4 Enrichment

Named Entity Recognition (NER) identifies information like artist names and historical periods, if not explicitly in metadata. Keyphrase extraction is also performed.

4.3.5 Vector Strategy (Chunking)

Required for the Vector (BERT) index due to token limits (usually 512). Long descriptions are split into smaller passages with small overlap (20 words) to prevent context loss at boundaries. Each chunk is embedded separately but links back to the parent artwork ID.

4.3.6 Embedding Generation

A BERT Transformer model is employed via the sentence-transformers library. The output is a 384 or 768-dimensional vector for every document/chunk.

4.3.7 Quality Gates

Ensure every artwork has a Title and ID; warn if descriptions or images are missing.

4.4 Indexing Design

- **Sparse Index:** A dictionary mapping terms to a postings list of document IDs. Furthermore, scoring data is used to support BM25F: doc_len (length of each document), avg_doc_len (average length across collection), and doc_freq (number of documents containing a specific term).
- **Fielded Indexing (BM25F):** Separate inverted indices allow for query-time boosting: Title (2.0 boost), Description (1.0 weight), and Artist (keyword exact match).
- **Dense Index:** The structure is centred around a dense matrix with a shape defined by the number of documents and their dimensions. The system employs an Exact k-

NN search algorithm that performs a brute-force dot product between the query vector and the entire document matrix.

- **Metadata Index (Filtering):** Implemented as an in-memory table (Python Dictionary or Pandas DataFrame). Operated using Boolean filters (Year, Medium, Location) applied post-ranking.

4.4.1 Update Strategy (Batch)

When the catalogue changes, the build script is re-run to regenerate index files from scratch.

4.5 Query Processing

Queries captured via the local interface undergo parallel processing:

4.5.1 Parsing and Normalisation

- **Sparse Pipeline:** Applies standard ingestion-level normalization (e.g., lowercasing, punctuation removal) to generate tokens for BM25F matching.
- **Semantic Pipeline:** Passes the raw natural language query directly to the embedding model. Stop-words are explicitly retained, as BERT relies on them for bidirectional context, outputting a 384-dimensional dense vector.

4.5.2 Post-Retrieval Filtering

To maintain indexing simplicity, the system utilizes a computationally inexpensive post-retrieval filtering strategy. The hybrid model initially retrieves a broad pool of 100 candidates. Items violating applied metadata constraints (e.g., medium, year) are discarded, and the remaining top 10 results are presented to the user.

4.6 Ranking and Fusion

4.6.1 Candidate Generation

The two retrieval models are executed in parallel to capture both exact matches and conceptual similarity.

- **Sparse Retrieval:** Executed using the BM25F algorithm.
- **Dense Retrieval:** Utilizes Exact k-NN based on Cosine Similarity to compare BERT-based query embeddings against the document collection. This allows the system to bridge conceptual gaps, such as matching a query for a "melancholy landscape" with an artwork described as a "gloomy countryside."

4.6.2 Score Normalisation & Fusion

Sparse and dense scores live on different scales. They are normalized using Min-Max Normalisation to make them comparable. The system utilizes a Weighted Linear Combination to merge the lists.

$$S_{norm} = \frac{S - S_{min}}{S_{max} - S_{min}}$$

4.6.3 Reranking (Optional)

Feature computation includes simple Boolean features (Title Match, Artist Match). Reranking logic is rule-based: apply multipliers (e.g., Title Match = x1.5) or penalties (e.g., No Image = x0.8).

4.6.4 Transparency

The system includes a "Why this result?" feature that details specific factors contributing to a document's rank, such as identified keyword matches and the degree of semantic similarity.

4.7 Error Handling & Fallback

To ensure system resilience, the search engine implements a "Fail Soft" policy:

- **Spelling correction:** Uses Levenstein Distance for suggestions against artist/medium dictionaries.
- **Vector Search Failure:** Fallback reverting to sparse-only search using BM25F.
- **Reranker Timeout:** If the optional cross-encoder exceeds a 500ms processing limit, the system skips the reranking stage to maintain low latency.
- **Logging:** All errors and automated fallbacks are recorded in a server log file.

4.8 Storage and Data Models

4.8.1 Core Stores

- **Raw Data:** ./data/raw/ (CSV catalogue and images).
- **Serialized Indices:** sparse_index.json, vectors.npy (NumPy binary), and documents.json (metadata for lookups).
- **Logs:** ./logs/server.log.

4.8.2 Versioning

The build script generates entirely new index files. Backups are created by renaming existing files with a date-stamped suffix before they are overwritten.

4.9 Quality and Evaluation

4.9.1 Offline Evaluation

Uses a domain-specific Golden Set of ~50 queries.

- **Known-Item:** "The Starry Night" (Must return ID: 105).
 - **Broad Topic:** "Impressionist landscapes" (Must return IDs: 101, 102...).
 - **Negative Test:** "Modernist cars" (Should return 0 results).
- Primary metrics are Precision@10, Recall, and Latency.

4.9.2 Continuous Evaluation

The system incorporates a regression test script (evaluate.py) executed prior to any code update. This script enforces strict performance thresholds regarding known-item accuracy and search latency.

5. Theoretical Basis for Retrieval Models

The architecture integrates two distinct mathematical foundations to address divergent search intents.

5.1 Probabilistic Lexical Model: BM25F

The search engine employs BM25F over classic BM25 due to the structured nature of an art gallery's catalogue (Title, Artist, Description). BM25F extends the probabilistic model to multiple weighted fields. Architectural advantages, as stated in Robertson et al. (2004), make it highly suitable for this search engine.

5.2 Vector Space Model: Exact k-NN via Cosine Similarity

The semantic pipeline utilizes a Sentence-BERT (SBERT) bi-encoder architecture to map unstructured descriptions into a high-dimensional continuous vector space (Reimers and Gurevych, 2019). Document relevance is determined by Cosine Similarity, which measures the angular distance between query $\mathbf{q}$ and document vector $\mathbf{d}$. This metric is specifically chosen for its length-invariance, ensuring that retrieval is based on semantic content rather than description magnitude.

$$\text{cosine}(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{|\mathbf{q}| |\mathbf{d}|}$$

For the 2,000-document corpus, the system employs an exact "brute-force" k -NN search. This is a strategic design choice: at this scale, the computational overhead is negligible, allowing the system to maintain 100% recall. This bypasses the inherent "recall-loss" trade-offs found in Approximate Nearest Neighbour (ANN) algorithms, such as HNSW or IVF, which are typically only required for million-scale datasets.

6. Evaluation Strategy

Retrieval effectiveness is evaluated using a domain-specific gold standard set of approximately 50 queries, stratified into known-item and exploratory semantic tasks. Known-item retrieval performance is measured using Mean Reciprocal Rank (MRR), defined as:

$$MRR = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{rank_i}$$

where $|Q|$ is the set of evaluation queries and $rank_i$ is the rank position of the first relevant document for query i . This metric rewards systems that return the target artwork at the highest possible rank position. Semantic retrieval performance is evaluated using Normalized Discounted Cumulative Gain (NDCG@10), calculated as:

$$NDCG_{10} = \frac{DCG_{10}}{IDCG_{10}}$$

where DCG_{10} represents the Discounted Cumulative Gain of the retrieved ranking up to position 10 and $IDCG_{10}$ represents the ideal ranking score. This metric penalizes relevant documents that appear lower in the ranked list while accounting for graded relevance.

Operational performance is assessed through query latency measurements, with a system target of sub-200 ms 95th-percentile latency. Benchmarking includes an initial warm-up query to mitigate model loading overhead, followed by repeated timed executions of each query to obtain stable latency measurements.

7. Team Workplan

Phase	Timeline	Deliverables (Data & Sparse)	Deliverables (Dense & Fusion)
1. Core Search	Weeks 1-2	Ingestion, ETL, and text parsing.	Sparse index (BM25F) and UI.
2. Hybrid Engine	Weeks 3-4	Vector generation and exact k -NN.	Weighted fusion and boosting.
3. Evaluation	Weeks 5-6	Golden set and offline metrics.	Tuning and final documentation.

8. Conclusion

This design delivers a modern, AI-enhanced search experience tailored for art gallery archives. Adhering to the strict engineering constraints of a single-developer academic project, the architecture combines BM25F for lexical precision with SBERT for semantic recall. Ultimately, this hybrid approach eliminates unnecessary operational overhead, allowing for a concentrated focus on robust search quality and user utility.

References

The architectural description presented in Section 4 is derived from a set of documents detailing the full design specification of the search engine, and the full technical specifications documents can be accessed at this link: https://github.qmul.ac.uk/ec25326/ir_cw_one

Devlin, J., Chang, M.W., Lee, K. and Toutanova, K. (2019) 'BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding', in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, Volume 1. Minneapolis: Association for Computational Linguistics, pp. 4171–4186.

Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D. and Yih, W.T. (2020) 'Dense Passage Retrieval for Open-Domain Question Answering', in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Online: Association for Computational Linguistics, pp. 6769–6780.

Reimers, N. and Gurevych, I. (2019) 'Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks', in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics.

Robertson, S.E., Walker, S., Hancock-Beaulieu, M.M. and Gatford, M. (1994) 'Okapi at TREC-3', in *Proceedings of the Third Text REtrieval Conference (TREC-3)*. Gaithersburg, MD: NIST, pp. 109–126.

Robertson, S., Zaragoza, H. and Taylor, M. (2004) 'Simple BM25 extension to multiple weighted fields', in *Proceedings of the thirteenth ACM international conference on Information and knowledge management (CIKM '04)*. New York: ACM, pp. 42–49.

Salton, G. and Buckley, C. (1988) 'Term-weighting approaches in automatic text retrieval', *Information Processing & Management*, 24(5), pp. 513–523.

Singhal, A., Buckley, C. and Mitra, M. (1996) 'Pivoted document length normalization', in *Proceedings of the 19th annual international ACM SIGIR conference on Research and development in information retrieval (SIGIR '96)*. New York: ACM, pp. 21–29.

Zhang, Q. (n.d.) *Information Retrieval Part 2: Indexing and TF-IDF*. [Lecture slides].

Zhang, Q. (n.d.) *Information Retrieval Models III: DfR and Pagerank*. [Lecture slides].